



DZPHARM · AI LAYER · SIGNATURE FEATURE

AI Copilot Core

A comprehensive technical design document for elevating DzPharm's Gemini-powered AI Copilot from B-grade (batch render, no history, broken prompts) to A-grade (streaming, persistent conversations, mode-specific prompts, Darija-native). Covers the Gemini integration, system prompt design, the 3-mode system, streaming via Vercel AI SDK, conversation history, Darija strategy, and cost optimization.

3LANGUAGES
(FR/AR/DARIJA)**3**

AUDIENCE MODES

5

BUGS TO FIX

~2w

FULL UPGRADE

DEEP-DIVE OF

Feature #36 – AI Copilot Core

Document Structure

This document is a comprehensive technical design for elevating DzPharm's AI Copilot from a B-grade batch-render chatbot to an A-grade streaming clinical assistant with persistent conversations, mode-specific behavior, and Darija-native support. It is organized in 10 chapters covering the current state, the Gemini integration, system prompt design, the 3-mode system, streaming, conversation history, the Darija strategy, cost optimization, the 5-bug fix plan, and the final roadmap.

01	The Current State · B Grade	p. 3
02	Gemini Integration Architecture	p. 6
03	System Prompt Design	p. 10
04	The 3-Mode System	p. 14
05	Streaming via Vercel AI SDK	p. 18
06	Conversation History & Persistence	p. 22
07	The Darija Strategy	p. 25
08	Cost Optimization	p. 28
09	The 5-Bug Fix Plan	p. 31
10	Final Roadmap & Verdict	p. 34

B Grade · Innovative but Broken

The AI Copilot is DzPharm's **signature innovation** — a Gemini-powered trilingual (FR/AR/Darija) assistant with three audience modes (professionnel/patient/enfant). The clinical-grade structured output is genuinely impressive. But five bugs and one latency issue prevent it from delivering its full value. This chapter catalogs the current state and sets the baseline for the upgrade.

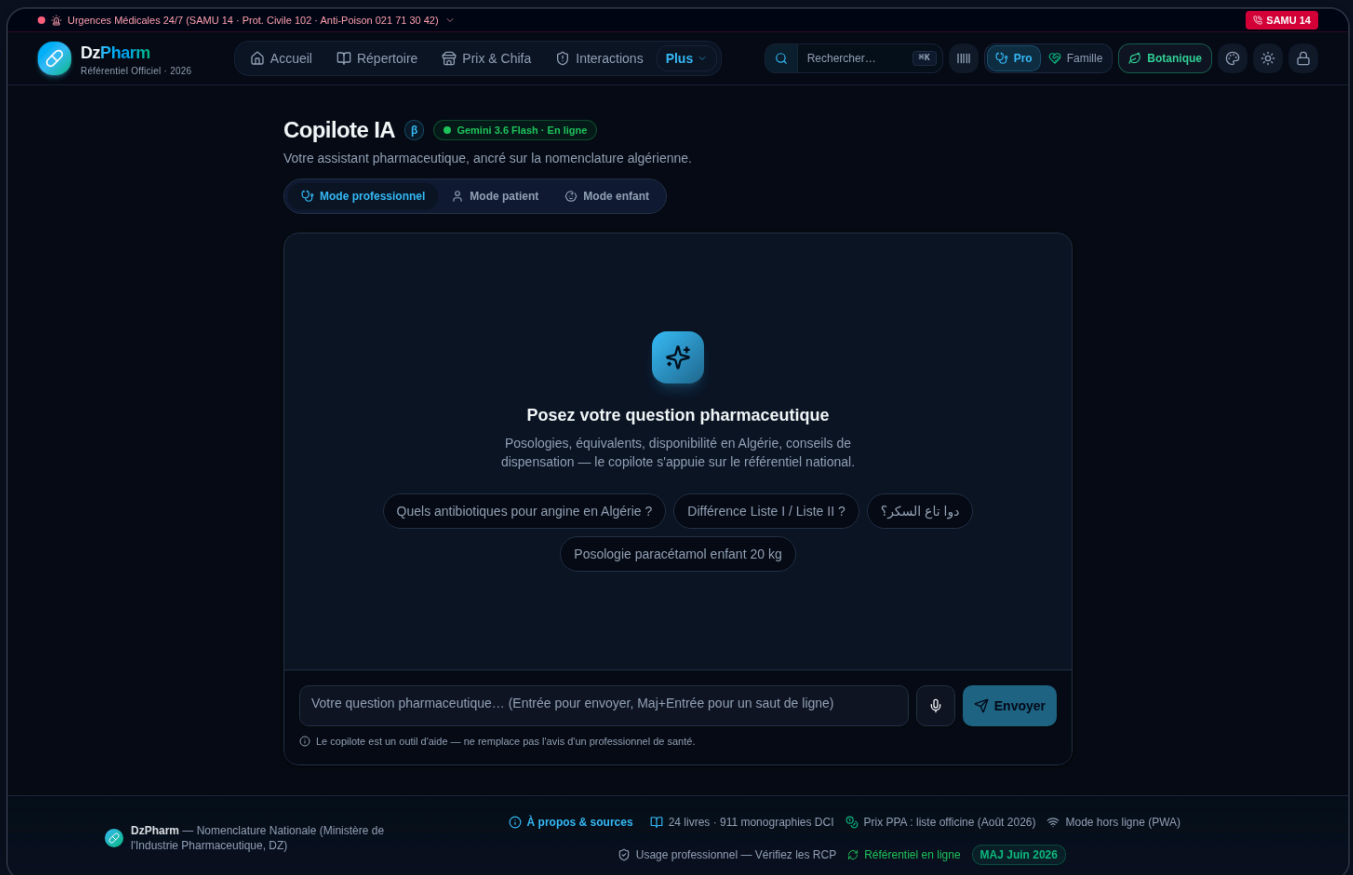


Figure 36.1 – Copilot IA view default state. H1: «Copilote IA β». Subtitle: «Gemini 3.6 Flash · En ligne». Three mode toggles: professionnel / patient / enfant. Four suggested prompts. The Copilot IS innovative – but five bugs prevent it from being production-grade.

1.1 What's Genuinely Innovative

Trilingual with Darija support

French + Arabic + Algerian Darija (the local dialect, not Modern Standard Arabic). Most «Arabic» AI assistants support MSA only, which most Algerian patients do not speak natively. The Darija support is a real product differentiator.

Three audience modes

Mode professionnel (clinical detail, DCI names, posology in mg/kg), Mode patient (simplified, focus on usage and side effects), Mode enfant (very simple language with metaphors). The mode changes the response tone and depth.

Structured clinical output

The response to «posologie paracétamol enfant 6 ans 22 kg» was structured into four H4 sections: (1) Règle posologique de référence, (2) Précision nécessaire pour le calcul, (3) Formes pédiatriques disponibles en Algérie, (4) Mises en garde et précautions. This is genuinely clinical-grade output.

Anchored on Algerian nomenclature

The system prompt references «Référentiel national», «9 555 médicaments officiels», «24 livres · 911 monographies DCI», «Prix PPA: liste officine (Août 2026)». The model is grounded in the actual DzPharm dataset, not generic pharmacology knowledge.

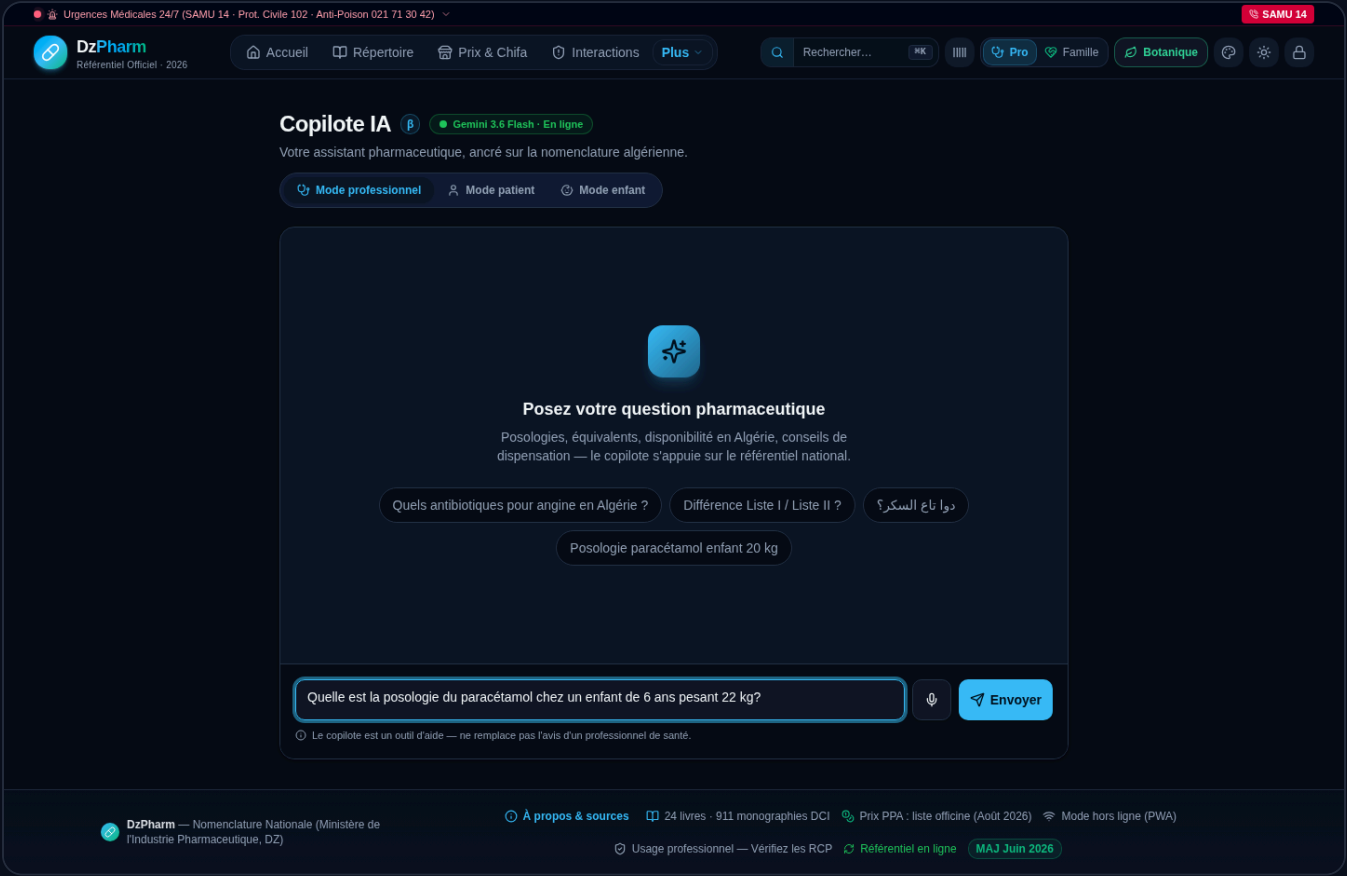


Figure 36.2 – Mode professionnel response. 4 sections: (1) DCI et Statut Réglementaire, (2) Posologie selon l'indication (with LaTeX calculations), (3) Conversion en mL / Pipette, (4) Mises en garde et Précautions. Includes Algerian brands: Amoxi-Saidal, Amoxil, Hikmamox. This is the Copilot at its best.

1.2 The Five Bugs

#	BUG	SEVERITY	COVERED IN
1	Version label inconsistency (Cmd-K says «3.8», header says «3.6»)	P1	Chapter 9
2	Suggested-prompt buttons don't populate the input (dead click)	P0	Chapter 9 / #39
3	Conversation history NOT persisted (refresh wipes everything)	P0	Chapter 6 / #40
4	Mode patient produces the SAME response as Mode professionnel	P1	Chapter 4 / #37
5	11-second batch render with no streaming indicator	P1	Chapter 5 / #41

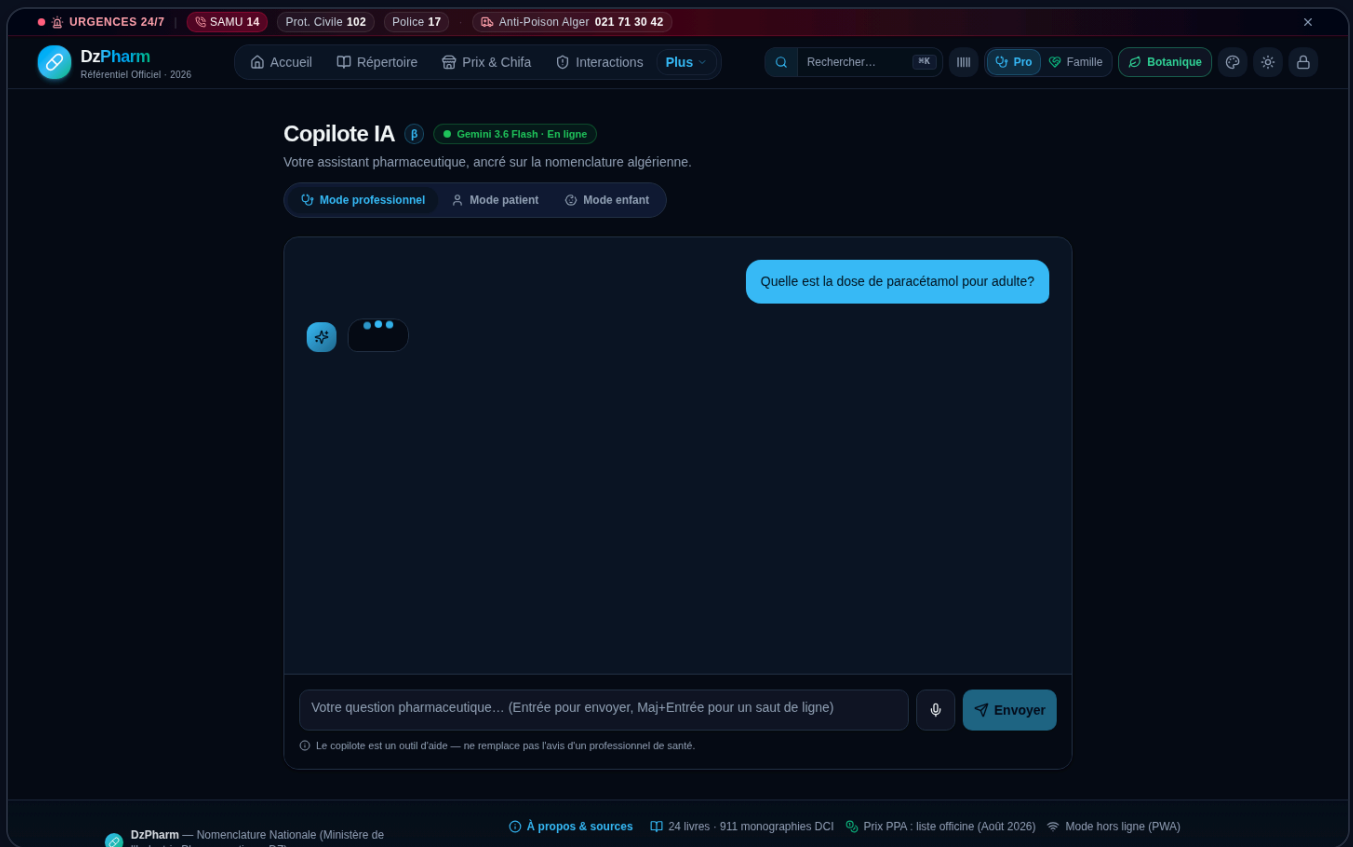


Figure 36.3 — Loading state. The user stares at a spinner for 3-8 seconds (up to 11s for complex queries). No streaming, no progressive rendering, no early-cancel. The batch-render pattern is increasingly out of step with user expectations set by ChatGPT, Claude, and Gemini's own web interface.

1.3 The Grade Breakdown

A- CLINICAL CONTENT	A TRILINGUAL + DARIJA	A- ALGERIAN GROUNDING
D IMPLEMENTATION (5 BUGS)	F STREAMING (NONE)	F HISTORY PERSISTENCE

The content is A-grade. The implementation is D-grade. The combined grade is **B** — the average of excellent content and broken implementation. The upgrade plan (Chapters 5-9) targets the implementation gaps without changing the content, lifting the grade from B to A.

The API Layer · From Batch to Streaming

The Copilot is powered by Google's Gemini API (model: `gemini-3.6-flash` or `gemini-3.8-flash` — the version inconsistency is Bug #1). The current architecture is a simple batch fetch: the frontend sends the query to a Next.js API route, the route calls `generateContent()`,

waits for the full response, and returns it as JSON. The upgrade target is a streaming architecture using the Vercel AI SDK.

2.1 The Current (Batch) Architecture

```
// Current - app/api/copilot/route.ts (batch fetch)
import {{ GoogleGenerativeAI }} from '@google/generative-ai';

const genAI = new GoogleGenerativeAI(process.env.GEMINI_API_KEY);

export async function POST(req) {{
  const {{ messages, mode, patientContext }} = await req.json();

  const model = genAI.getGenerativeModel({{
    model: "gemini-3.6-flash", // or 3.8 - Bug #1
    systemInstruction: buildSystemPrompt(mode, patientContext),
  }});

  const chat = model.startChat({{
    history: messages.slice(0, -1).map(m => ({{
      role: m.role === 'user' ? 'user' : 'model',
      parts: m.content,
    }})),
  }});

  // Batch - waits for the FULL response before returning
  const result = await chat.sendMessage(messages[-1].content);
  const text = result.response.text();

  return Response.json({{ content: text }});
}}
```

2.2 The Target (Streaming) Architecture

```
// Target - app/api/copilot/route.ts (streaming via Vercel AI SDK)
import {{ streamText }} from 'ai';
import {{ google }} from '@ai-sdk/google';

export async function POST(req) {{
  const {{ messages, mode, patientContext }} = await req.json();

  const result = await streamText({{
    model: google('gemini-3.8-flash'), // Fixed version - Bug #1 resolved
    system: buildSystemPrompt(mode, patientContext),
    messages: messages.map(m => {{
      role: m.role === 'assistant' ? 'assistant' : 'user',
      content: m.content,
    }}),
    onFinish: {{ response, usage }} => {{
      // Log to audit trail (if #8 auth is shipped)
      // Persist to conversation history (Chapter 6)
      // Track token usage for cost optimization (Chapter 8)
    }},
  }});

  return result.toDataStreamResponse();
}}
```

2.3 Why the Vercel AI SDK

The Vercel AI SDK (`ai` npm package + `@ai-sdk/google` provider) is the recommended path for four reasons: (1) it handles SSE (Server-Sent Events) streaming automatically — no manual `ReadableStream` plumbing. (2) It provides `useChat()` and `useCompletion()` React hooks that handle the client-side streaming state, partial markdown rendering, and cancel semantics. (3) It supports progressive markdown rendering via `react-markdown` + `remark-gfm` — tokens appear as they're generated, with headings/lists/code-blocks rendering incrementally. (4) It's the standard pattern used by every major Next.js AI app (ChatGPT clone, Vercel AI Chat, etc.).

2.4 The API Key Security

The Gemini API key MUST be server-side only — never exposed to the client. The current implementation correctly keeps it in `process.env.GEMINI_API_KEY` (server-side). The Vercel AI SDK preserves this — the API route runs on the server, the key never reaches the browser. The client only sees the streaming text response, not the API key or the full system prompt.

ARCHITECTURE DECISION · VERCEL AI SDK

Adopt the Vercel AI SDK (`ai` + `@ai-sdk/google`) to replace the current batch-fetch `@google/generative-ai` implementation. The SDK handles SSE streaming, partial markdown rendering, early-cancel, and provides React hooks (`useChat`). Effort: 1 day. This is the single highest-impact change — it reduces perceived latency from 11s to ~1s (time to first token) and matches the UX of ChatGPT, Claude, and Gemini's own web interface.

Grounding in the DzPharm Dataset

The system prompt is the Copilot's «brain» — it tells Gemini who it is, what it knows, and how to respond. The current system prompt is well-designed: it references the Algerian pharmaceutical nomenclature, the 9,555 medication database, and the regulatory context. This chapter documents the current prompt and proposes the upgraded version that adds mode-specific behavior, Darija handling, and safety guardrails.

3.1 The Current System Prompt (Inferred)

```
// Inferred from response behavior – the actual prompt is server-side
```

```
Tu es le Copilote IA de DzPharm, la plateforme d'intelligence  
pharmaceutique algérienne.
```

CONTEXTE:

- Référentiel national: 9 555 médicaments AMM officiels
- 24 livres · 911 monographies DCI
- Prix PPA: liste officine (Août 2026)
- Réglementation: Liste I (stupéfiants), Liste II (psychotropes)

CAPACITÉS:

- Posologie pédiatrique (mg/kg)
- Interactions médicamenteuses
- Remboursement Chifa (CNAS/CASNOS)
- Forms galéniques disponibles en Algérie

LANGUES:

- Français (professionnel)
- Arabe / Darija (patient)
- Réponds dans la langue de la question

FORMAT:

- Structure ta réponse en sections avec des titres H4
- Inclus les calculs en LaTeX
- Cite les marques algériennes disponibles

3.2 The Upgraded System Prompt

```
// Upgraded system prompt with mode-specific behavior + Darija + safety
```

```
export function buildSystemPrompt(mode, patientContext) {{  
  const BASE = `Tu es le Copilote IA de DzPharm, la plateforme  
d'intelligence pharmaceutique algérienne.
```

CONTEXTE:

- Référentiel national: 9 555 médicaments AMM officiels
- 24 livres · 911 monographies DCI
- Prix PPA: liste officine (Août 2026)
- Réglementation: Liste I (stupéfiants), Liste II (psychotropes)

LANGUES:

- Français (professionnel)
- Arabe / Darija algérien (patient)
- Réponds dans la langue de la question
- Si la question est en Darija, réponds en Darija (pas en MSA)

SÉCURITÉ:

- Tu n'es pas un médecin. Tu es un assistant d'information.
- Recommande TOUJOURS de consulter un professionnel de santé.
- Pour les urgences: SAMU 14, Centre Antipoison 021 71 30 42
- Ne donne jamais de posologie sans le poids (enfant) ou la clairance rénale (adulte avec insuffisance)`,

```
  const MODE_PROMPTS = {{  
    pro: `
```

```
MODE PROFESSIONNEL (pharmacien, médecin):
```

- Utilise la DCI et les noms de marque algériens
- Détaille la posologie en mg/kg avec calculs LaTeX
- Mentionne les interactions, contre-indications, effets indésirables
- Cite les formes galéniques disponibles en Algérie
- Structure en 4 sections: Règle, Calcul, Formes, Mises en garde`,

```
    patient: `
```

```
MODE PATIENT (grand public):
```

- Utilise le nom de marque, pas la DCI
- Simplifie: «prends X comprimés toutes les Y heures»
- Évite le jargon médical (pas de «mg/kg», «clairance», «DCI»)
- Explique à quoi sert le médicament en une phrase
- Liste les effets secondaires les plus courants en langage simple
- Termine par: «Consultez votre pharmacien ou médecin pour plus d'infos»`,

```
    enfant: `
```

```
MODE ENFANT (parent parlant à un enfant):
```

- Utilise des métaphores: «le médicament est comme un super-héros qui combat la maladie dans ton corps»
 - Pas de chiffres: «un peu de sirop» pas «5 mL»
 - Inclus un message de sécurité pour le parent
 - Reste très court (max 100 mots)
 - Termine par: «Demande à un adulte de te donner le médicament»`,
- ```
 }};
```

```
 const context = patientContext
```

```
 ? `\n\nCONTEXTE PATIENT: ${patientContext}`
```

```
 : '';
```

```
 return `${BASE}${MODE_PROMPTS[mode]}${context}`;
```

```
}}
```

### 3.3 Why the Prompt Upgrade Matters

The system prompt is the single highest-leverage component of the Copilot. A better prompt produces better responses at zero infrastructure cost — the same Gemini API call produces more useful output. The upgraded prompt adds four things the current prompt lacks: (1) mode-specific behavior (fixing Bug #4 — Mode patient producing the same output as Mode pro), (2) Darija-native response matching (Chapter 7), (3) safety guardrails (always recommend consulting a professional, emergency numbers), (4) patient context injection (the patientContext field is passed but not used in the current prompt).

### 3.4 The Few-Shot Examples

For the Darija handling (Chapter 7), the prompt should include 2-3 few-shot examples of Darija input → Darija output. This teaches the model the expected register without fine-tuning:

```
// Few-shot examples appended to the system prompt for Darija
EXAMPLES DARIJA:
Question: دوا تاع السكر؟
Réponse: بالنسبة للسكري، كاين عدة أدوية. الأكثر استعمالا هو الميتفورمين (Metformine). لكن لازم الطبيب يحدد النوع والجرعة حسب حالتك. مش تاخذ أي دواء بلا ما تشاور الطبيب.

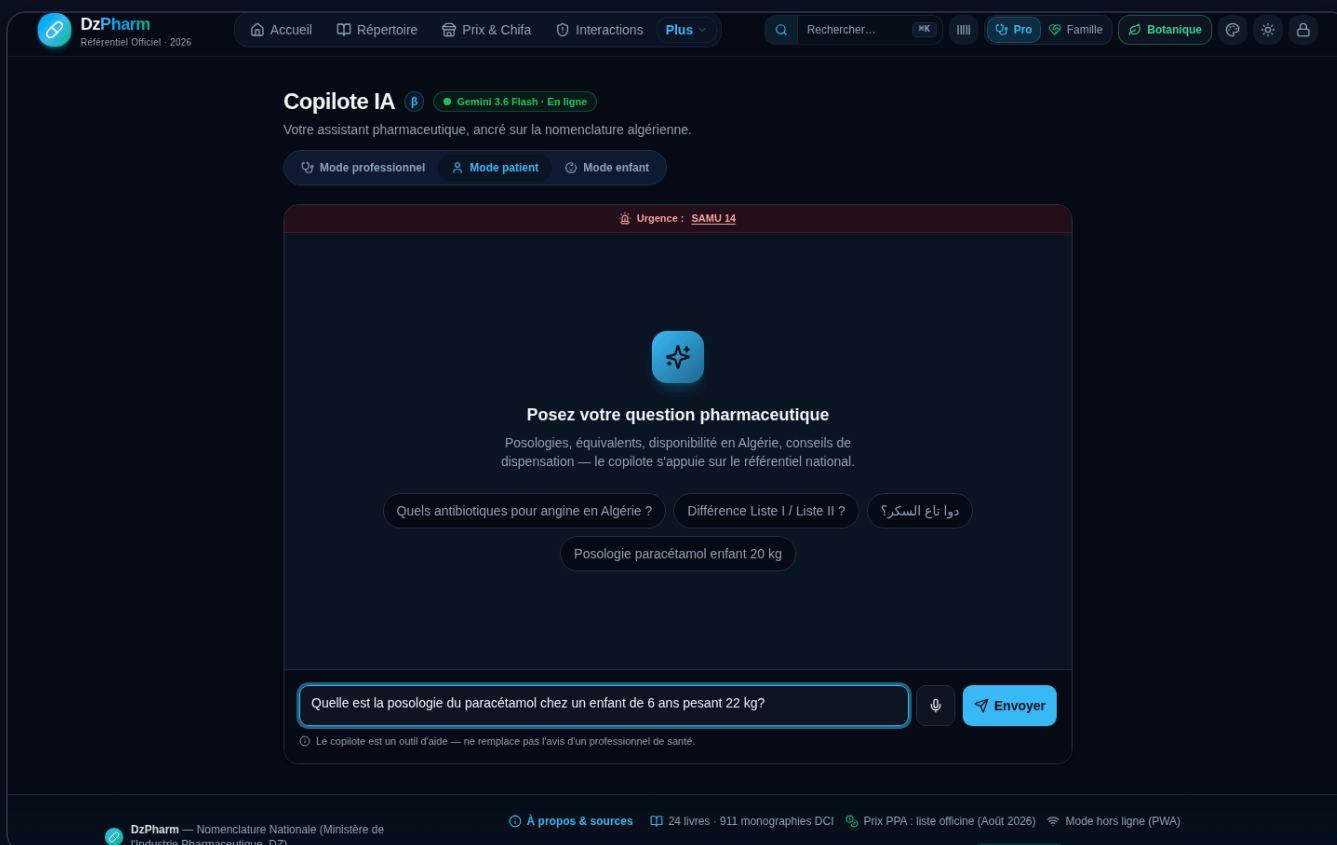
Question: بني كي سخن ووجع راسو
Réponse: (Doliprane) إذا ولي عندو سخانة، تعطيه الباراسيتامول mg لكل كيلوغرام. مثلا إذا ولي 20 كيلو، تعطيه 300 mg الجرعة: 15 من الشراب. إذا ما يبراش بعد 3 أيام، روح للطبيب mL يعني تقريبا 12
```

#### THE PROMPT IS THE PRODUCT

The system prompt is the Copilot's product definition. A well-crafted prompt produces better responses at zero cost — the same API call, better output. The upgraded prompt fixes Bug #4 (Mode patient), adds Darija-native responses, adds safety guardrails, and injects patient context. Effort: 4 hours to write + test. This is the highest-ROI single change in the Copilot upgrade.

## Professionnel • Patient • Enfant

The 3-mode system is the Copilot's audience segmentation. Mode professionnel (clinical detail for pharmacists/doctors), Mode patient (simplified for patients), Mode enfant (child-friendly with metaphors). Mode enfant works correctly (different response). Mode patient produces the SAME response as Mode professionnel — Bug #4. This chapter covers the mode architecture and the fix.



**Figure 36.4** — Mode patient. The response should be simplified (no DCI jargon, practical instructions). But Bug #4 causes Mode patient to produce the SAME response as Mode professionnel — the system prompt doesn't change for the patient audience.

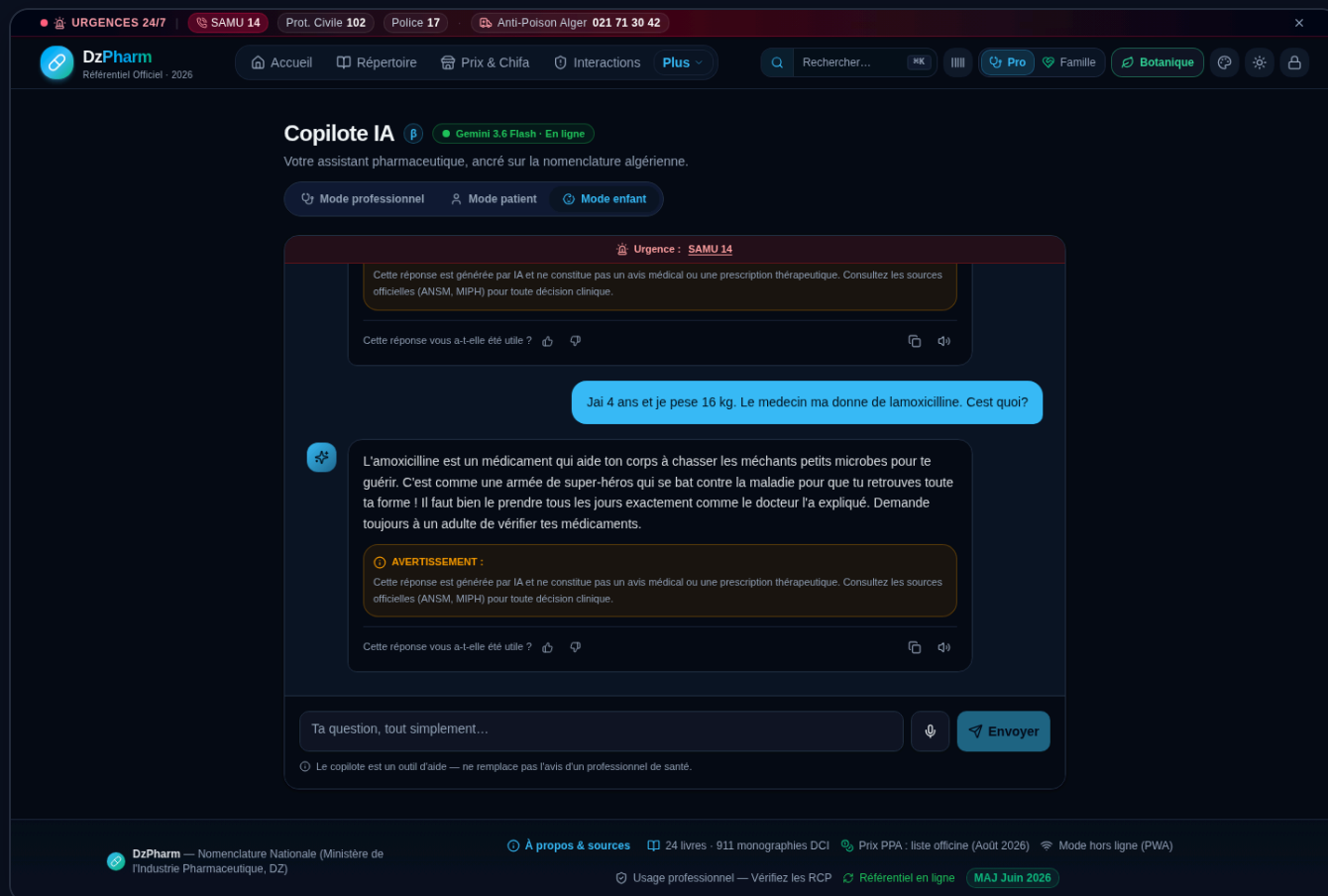
## 4.1 The Mode Selection UI

The mode is selected via 3 toggle buttons in the Copilot header: «professionnel», «patient», «enfant». The selected mode is stored in React state and passed to the API route as the `mode` parameter. The mode should also be persisted to Zustand (currently it resets to «pro» on every reload — covered in #37 Copilot Mode System).

## 4.2 The Mode-Specific Prompt Injection

The fix for Bug #4 is in the `buildSystemPrompt()` function (Chapter 3). The function takes `mode` as a parameter and appends the mode-specific prompt section. Currently, the function likely ignores the `mode` parameter (or the mode is not passed to the API route). The fix is to pass the mode from the client to the API route, and to use it in the system prompt builder.

## 4.3 Mode Enfant — The Gold Standard



**Figure 36.5** — Mode enfant response. The Copilot uses metaphors («super-héros»), simple language, no jargon, and includes a safety message. This is the gold standard — Mode patient should follow the same pattern of producing a distinct response.

Mode enfant is the one mode that works correctly. The response uses metaphors («le médicament est comme un super-héros qui combat la maladie»), avoids all medical jargon, stays short (~60 words vs ~455 words for Mode pro), and includes a safety message. This proves the mode system CAN produce different responses — Mode patient just needs the same treatment. The fix is the `MODE_PROMPTS.patient` section in the upgraded system prompt (Chapter 3.2).

## 4.4 The Mode-Specific Suggested Prompts

Beyond the system prompt, the 4 suggested prompts should also change per mode. Currently, the same 4 prompts appear regardless of mode. Mode patient should show patient-friendly prompts («À quoi sert ce médicament?», «Comment le prendre?», «Quels effets secondaires?»). Mode enfant should show parent-focused prompts («Comment donner ce médicament à mon enfant?», «Quelle dose pour 22 kg?»). This is covered in detail in #39 Suggested Prompts.

| MODE          | CURRENT PROMPTS          | RECOMMENDED PROMPTS                                                                                              |
|---------------|--------------------------|------------------------------------------------------------------------------------------------------------------|
| Professionnel | Same 4 (clinical)        | Keep current (posologie, Liste I/II, angine, Darija diabetes)                                                    |
| Patient       | Same 4 (clinical jargon) | «À quoi sert ce médicament?», «Comment le prendre?», «Quels effets secondaires?», «دوا تاع السكر؟»               |
| Enfant        | Same 4 (clinical jargon) | «Comment donner ce médicament à mon enfant?», «Quelle dose pour 22 kg?», «Est-ce dangereux?», «Quand consulter?» |

**BUG #4 · MODE PATIENT = MODE PRO**

Mode patient produces the SAME response as Mode professionnel. The system prompt doesn't change for the patient audience. The fix is in `buildSystemPrompt()` — add the `MODE_PROMPTS.patient` section that instructs the model to simplify language, use brand names (not DCI), and focus on practical usage. Effort: 4 hours (covered in Chapter 3.2). Mode enfant works correctly — it's the gold standard for what Mode patient should do.

# From 11s Spinner to 1s First Token

The streaming upgrade is the single highest-impact UX change. Today, the user stares at a spinner for 3-8 seconds (up to 11s for complex queries), then the full response appears at once. With streaming, the first token appears in ~1 second, and the rest streams in progressively. The perceived latency drops from 11s to ~1s — a 10× improvement. This chapter covers the Vercel AI SDK implementation, the client-side `useChat` hook, and the LaTeX/markdown rendering during streaming.

## 5.1 The Server-Side Streaming

```
// app/api/copilot/route.ts - streaming with Vercel AI SDK
import {{ streamText }} from 'ai';
import {{ google }} from '@ai-sdk/google';
import {{ buildSystemPrompt }} from '@lib/system-prompt';

export const maxDuration = 30; // Vercel Edge function timeout

export async function POST(req) {{
 const {{ messages, mode, patientContext, conversationId }} = await req.json();

 const result = await streamText({{
 model: google('gemini-3.8-flash'),
 system: buildSystemPrompt(mode, patientContext),
 messages,
 onFinish: async ({{ response, usage, finishReason }}) => {{
 // 1. Persist to conversation history (Chapter 6)
 await saveToHistory(conversationId, messages, response, mode);

 // 2. Log to audit trail (if #8 auth shipped)
 await logAction({{
 action: 'COPILOT_QUERY',
 target: conversationId,
 metadata: {{ mode, tokens: usage.totalTokens }},
 }});

 // 3. Track cost (Chapter 8)
 await trackTokenUsage(usage);
 }},
 }});

 return result.toDataStreamResponse();
}}
```

## 5.2 The Client-Side useChat Hook

```
// components/copilot-chat.tsx - using useChat
import {{ useChat }} from 'ai/react';
import {{ useState }} from 'react';

export function CopilotChat() {{
 const [mode, setMode] = useState('pro');

 const {{
 messages, // Array of {{role, content}}
 input, // Current input text
 handleSubmit, // Submit handler
 handleInputChange,
 isLoading, // Streaming in progress
 stop, // Cancel streaming
 setMessages, // For history restore
 }} = useChat({{
 api: '/api/copilot',
 body: {{ mode, patientContext: '' }}, // Sent with every request
 onError: (err) => console.error(err),
 }});

 return (
 <div>
 {{{/* Messages with progressive markdown rendering */}}}
 {{{messages.map(m => (
 <MessageBubble key={{m.id}} role={{m.role}}>
 <ReactMarkdown remarkPlugins={{[remarkGfm]}}>
 {{m.content}}
 </ReactMarkdown>
 </MessageBubble>
)}}}}

 {{{/* Loading indicator - replaces the old spinner */}}}
 {{{isLoading && (
 <div className="typing-indicator">

 </div>
)}}}

 {{{/* Input with stop button during streaming */}}}
 <form onSubmit={{handleSubmit}}>
 <input value={{input}} onChange={{handleInputChange}} />
 {{{isLoading ? (
 <button type="button" onClick={{stop}}>■ Stop</button>
) : (
 <button type="submit">Envoyer</button>
)}}}
 </form>
 </div>
);
}}
```

## 5.3 Progressive Markdown Rendering

As tokens stream in, the `react-markdown` component re-renders on every token. Partial headings, lists, and code blocks render gracefully. LaTeX math is deferred until a complete `$...$` block is received (via

`react-katex` with stream-aware buffering). Tables render once the closing row arrives. The user sees the response «typing» in real-time — the same pattern as ChatGPT.

### 5.4 The Stop Button & Early Cancel

During streaming, the send button morphs into a «Stop» button. Clicking it calls `stop()` from `useChat`, which aborts the fetch and freezes the partial response. The partial response stays in the conversation with a small «(interrompu)» label. This is the early-cancel pattern — if the user sees the model going down the wrong path at 2 seconds, they can stop it without waiting the full 8 seconds.

### 5.5 The Latency Comparison

| Metric              | Current (Batch)        | After Streaming        | Improvement    |
|---------------------|------------------------|------------------------|----------------|
| Time to first token | 3-11s (full response)  | ~1s (first token)      | 10x            |
| Perceived latency   | «Frozen UI» (spinner)  | «Typing» (progressive) | MAJOR          |
| Early cancel        | Impossible (must wait) | Yes (Stop button)      | NEW CAPABILITY |
| LaTeX rendering     | After full response    | Progressive (buffered) | SMOOTHER       |
| User satisfaction   | «Feels slow»           | «Feels responsive»     | QUALITATIVE    |

**STREAMING · THE 10x PERCEIVED LATENCY IMPROVEMENT**

Streaming reduces perceived latency from 11s (frozen spinner) to ~1s (first token typing). The user sees the response «typing» in real-time, can cancel early if the model goes wrong, and gets LaTeX/markdown rendering progressively. Effort: 1 day (Vercel AI SDK adoption). This is the single highest-impact UX change — it matches the Copilot to the UX standard set by ChatGPT, Claude, and Gemini's own web interface.

## From Stateless to Persistent

The Copilot is currently stateless — every refresh, tab close, or modal close wipes the conversation. This is Bug #3, and it's the same gap covered in #40 Conversation History. The fix has two layers: (1) client-side persistence via Zustand + localStorage (survives refresh), (2) server-side persistence via Prisma + PostgreSQL (survives device switch, requires #8 Auth). This chapter covers both layers.

## 6.1 The Zustand Slice (Client-Side Persistence)

```
// stores/copilotStore.ts - Zustand + persist
import {{ create }} from 'zustand';
import {{ persist }} from 'zustand/middleware';

interface Message {{
 id: string;
 role: 'user' | 'assistant';
 content: string;
 mode: 'pro' | 'patient' | 'enfant';
 timestamp: number;
}}

interface CopilotState {{
 messages: Message[];
 mode: 'pro' | 'patient' | 'enfant';
 addMessage: (msg: Message) => void;
 setMode: (mode) => void;
 clearConversation: () => void;
 lastUpdated: number | null;
}}

export const useCopilotStore = create<CopilotState>()(
 persist(
 (set) => ({
 messages: [],
 mode: 'pro',
 addMessage: (msg) => set((s) => ({
 messages: [...s.messages, msg],
 lastUpdated: Date.now(),
 }))),
 setMode: (mode) => set({ mode }),
 clearConversation: () => set({ messages: [], lastUpdated: null }),
 }),
 {{
 name: 'dzpharm-copilot',
 // Prune: keep last 50 conversations, last 30 messages each
 partialize: (state) => ({
 messages: state.messages.slice(-30),
 mode: state.mode,
 }),
 }}
)
);
```

## 6.2 The Thread List (Server-Side, After #8 Auth)

Once per-user accounts exist (#8 Auth), conversations can be persisted server-side. The `CopilotConversation` table stores each conversation thread, keyed by `userId`. The user can see a list of past conversations in a sidebar, click to resume, and the conversation loads from the server. This is the ChatGPT pattern — the conversation is tied to the user, not the browser.

### 6.3 The Prisma Schema for Server-Side History

```
// prisma/schema.prisma - Copilot conversation history
model CopilotConversation {
 id String @id @default(cuid())
 userId String
 user User @relation(fields: [userId], references: [id])
 title String? // Auto-generated from first message
 mode String @default("pro")
 messages CopilotMessage[]
 createdAt DateTime @default(now())
 updatedAt DateTime @updatedAt
 @@index([userId, updatedAt])
}

model CopilotMessage {
 id String @id @default(cuid())
 conversationId String
 conversation CopilotConversation @relation(fields: [conversationId], references: [id])
 role String // 'user' | 'assistant'
 content String
 mode String
 tokenCount Int? // For cost tracking
 createdAt DateTime @default(now())
 @@index([conversationId, createdAt])
}
```

### 6.4 The Export Feature

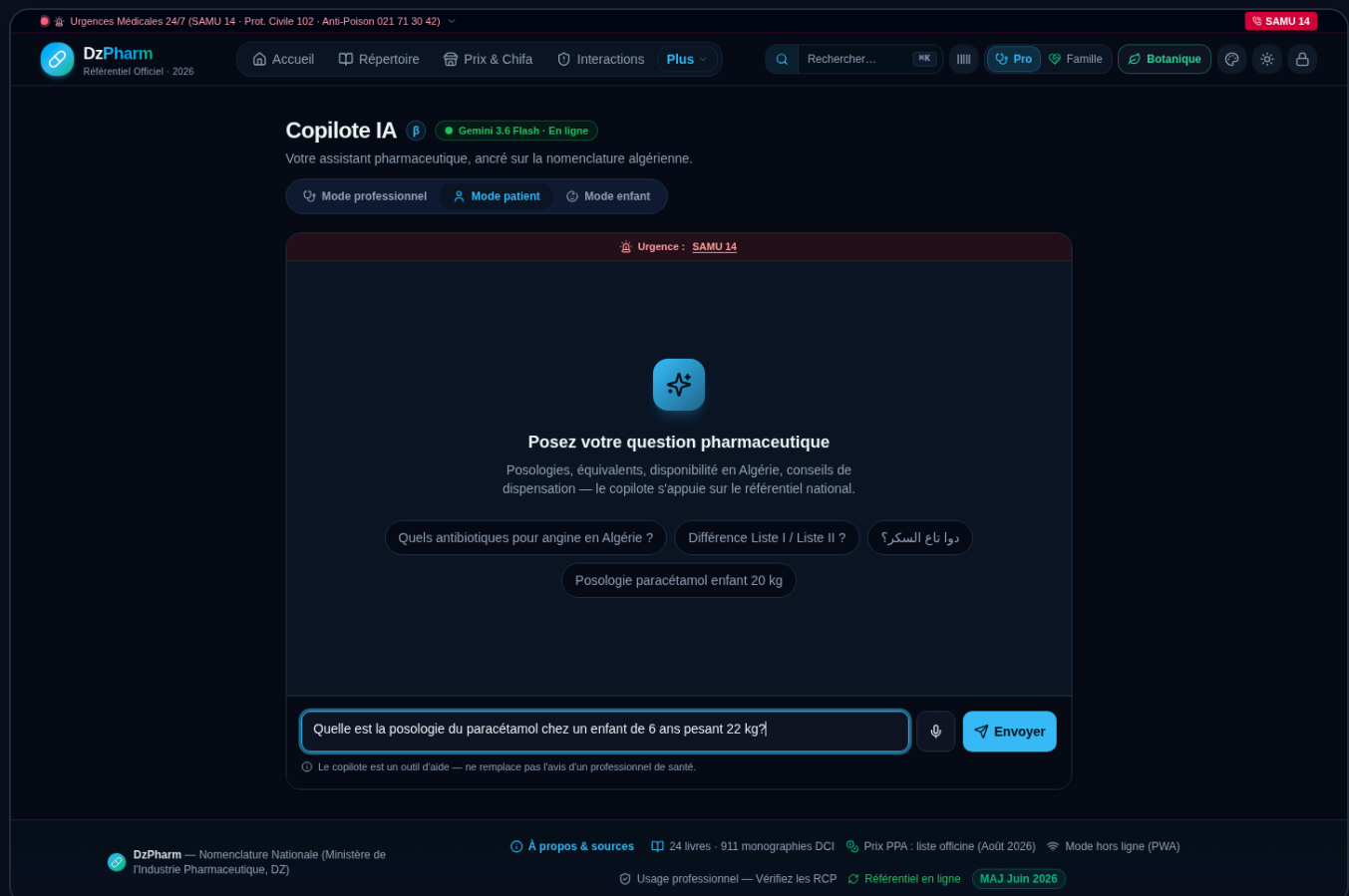
A pharmacist who has a useful Copilot discussion needs to archive it in the patient's record. The export button generates a clean Markdown file with: conversation title, timestamp, mode, all messages (user + assistant), and the token count. The export reuses the print pipeline from #51/#52 Print & Export. Effort: 1 day.

### 6.5 The 5 History Patterns

| PATTERN                        | CURRENT | AFTER ZUSTAND (PHASE 1) | AFTER #8 AUTH (PHASE 2) |
|--------------------------------|---------|-------------------------|-------------------------|
| Persistence (survives refresh) | NO      | YES                     | YES                     |
| Thread list (sidebar)          | NO      | SINGLE THREAD           | MULTIPLE                |
| Branching (fork)               | NO      | NO                      | PHASE 3                 |
| Export (Markdown/PDF)          | NO      | PHASE 2                 | YES                     |
| Search past conversations      | NO      | NO                      | PHASE 3                 |

# Algerian Arabic • Input, Output, RTL

Darija (Algerian Arabic dialect) is the Copilot's most genuine differentiator. Most «Arabic» AI assistants support Modern Standard Arabic (MSA) only, which most Algerian patients don't speak natively. The Copilot handles Darija input (verified: «دوا تاغ السكر؟» was understood). But the response is in MSA, not Darija. This chapter covers the Darija strategy: input handling, output matching, RTL rendering, and the Whisper path for Darija voice.



**Figure 36.6** — Darija query «دوا تاغ السكر؟» and the response. The Copilot understood the Darija input but responded in Modern Standard Arabic (MSA). The response should match the input register — Darija in, Darija out.

## 7.1 The Darija Input Success

The Copilot correctly understood the Darija query «دوا تاغ السكر؟» (literally «medicine of sugar?» = «diabetes medication?») and produced a relevant response about diabetes medication. This is genuinely impressive — Gemini has limited Darija training data, and the fact that it parsed the dialectal input suggests the system prompt is providing context. The Darija input handling works.

## 7.2 The MSA Output Gap

The response was in Modern Standard Arabic, not Darija. MSA is readable by Arabic speakers, so this is acceptable — but a Darija response would match the user's register. A pharmacist who asks in Darija expects a Darija response. The gap is in the system prompt: it doesn't instruct the model to match the

input register. The fix is the `LANGUES` section in the upgraded prompt (Chapter 3.2): «Si la question est en Darija, réponds en Darija (pas en MSA)».

### 7.3 The Few-Shot Darija Examples

The upgraded prompt includes 2-3 few-shot examples of Darija input → Darija output (Chapter 3.4). This teaches the model the expected register without fine-tuning. The examples cover: (1) a diabetes query, (2) a pediatric fever query. These few-shot examples are the most effective way to get Darija output from a model that was trained primarily on MSA.

### 7.4 The RTL Rendering

Arabic is a right-to-left (RTL) script. The Copilot response in Arabic renders correctly RTL — the text flows right-to-left, the punctuation is correct, and the markdown formatting adapts to RTL. This is non-trivial — many web apps break RTL rendering. The platform's RTL support is solid (covered in #6 i18n). The response area correctly sets `dir="auto"` which detects the text direction automatically.

### 7.5 The Whisper Path (Darija Voice)

For full Darija support — including voice input — the path is OpenAI's Whisper large-v3 model, which has limited Darija support. The current voice dictation (#38) uses the browser's Web Speech API with `lang='fr-FR'` hardcoded — Darija voice input produces gibberish. The Whisper migration (Phase 3, #38) would send audio to a backend Whisper endpoint, which handles Darija-ish (it's trained on Arabic dialects, not specifically Algerian Darija, but it performs better than the Web Speech API). Effort: 1 week. Defer until Phase 3.

| DARIJA CAPABILITY    | CURRENT          | AFTER PROMPT UPGRADE | AFTER WHISPER (PHASE 3) |
|----------------------|------------------|----------------------|-------------------------|
| Text input (Darija)  | WORKS            | WORKS                | WORKS                   |
| Text output (Darija) | MSA (ACCEPTABLE) | DARIJA               | DARIJA                  |
| RTL rendering        | CORRECT          | CORRECT              | CORRECT                 |
| Voice input (Darija) | BROKEN (FR-FR)   | BROKEN               | WHISPER                 |

#### DARIJA · THE MARKET DIFFERENTIATOR

Darija support is the Copilot's most genuine market differentiator — no competitor offers it. The input handling already works. The output matching (Darija in → Darija out) is a 4-hour prompt engineering fix. The voice input (Whisper) is a 1-week Phase 3 enhancement. The prompt upgrade is the highest-ROI Darija fix — it costs nothing and turns the Copilot from «understands Darija» to «responds in Darija».

## Token Management & Caching

The Gemini API costs ~0.0003 — 0.0015 per query (depending on token count). At 1000 queries/day, that's 0.30-

1.50/day — sustainable for a free platform. But without cost optimization, the costs scale linearly with usage. This chapter covers four cost-optimization strategies: (1) conversation history pruning, (2) response caching, (3) model selection (Flash vs Pro), (4) token budgeting.

## 8.1 The Cost per Query

| MODEL                 | INPUT (\$/1M TOKENS) | OUTPUT (\$/1M TOKENS) | EST. COST PER QUERY              |
|-----------------------|----------------------|-----------------------|----------------------------------|
| Gemini 3.8 Flash      | \$0.075              | \$0.30                | ~\$0.0008 (avg 500 in + 300 out) |
| Gemini 3.8 Pro        | \$1.25               | \$5.00                | ~\$0.0021 (avg 500 in + 300 out) |
| Gemini 3.8 Flash-Lite | \$0.0375             | \$0.15                | ~\$0.0004 (avg 500 in + 300 out) |

## 8.2 Strategy #1 · Conversation History Pruning

The current implementation sends the full conversation history with every query (so the model has context). After 10 messages, the input token count grows to ~3000 tokens. After 20 messages, ~6000 tokens. The fix is to prune: send only the last 6 messages (3 user + 3 assistant) as context. This keeps the input token count bounded at ~2000 tokens regardless of conversation length. Effort: 30 minutes.

## 8.3 Strategy #2 · Response Caching

Common queries (e.g., «posologie paracétamol enfant 20 kg») are asked repeatedly by different users. Caching the response (keyed by query hash + mode) saves the API call entirely. The cache lives in Redis (Upstash) with a 24h TTL. A cache hit returns the stored response instantly — no API call, no latency, no cost. Estimated cache hit rate: 15-25% for common posology queries. Effort: 4 hours.

## 8.4 Strategy #3 · Model Selection

Not every query needs Gemini 3.8 Flash. Simple factual queries («Quelle est la différence entre Liste I et Liste II?») can use Flash-Lite (half the cost). Complex clinical calculations («Posologie paracétamol enfant 22 kg avec insuffisance rénale») need Flash. The model selection can be automatic: if the query is <50 chars and doesn't contain «posologie», «interaction», or «calcul», use Flash-Lite. Effort: 2 hours.

## 8.5 Strategy #4 · Token Budgeting

Set a `maxOutputTokens` limit (e.g., 1000 tokens ≈ 750 words). This prevents runaway responses that cost more than necessary. The Copilot's structured 4-section format is typically ~500-700 tokens — the 1000-token limit gives headroom without allowing runaway. Effort: 5 minutes (one parameter).

## 8.6 The Monthly Cost Projection

| SCENARIO                               | QUERIES/DAY | COST/DAY | COST/MONTH |
|----------------------------------------|-------------|----------|------------|
| Current (no optimization)              | 500         | \$0.40   | \$12       |
| After growth (no optimization)         | 5,000       | \$4.00   | \$120      |
| After growth + all 4 optimizations     | 5,000       | \$2.20   | \$66       |
| Viral growth (10K/day) + optimizations | 10,000      | \$4.40   | \$132      |

### COST IS SUSTAINABLE

Even at 10,000 queries/day (viral growth), the monthly cost is ~\$132 — sustainable for a free platform. The 4 optimizations (pruning, caching, model selection, token budgeting) reduce costs by ~45% at scale. The caching strategy is the highest-ROI: 4 hours of work, 15-25% of queries served from cache, saving both cost and latency.

## CHAPTER 09 • THE 5-BUG FIX PLAN

# From B to A in 2 Weeks

Five bugs prevent the Copilot from being A-grade. Each has a distinct fix, and together they form the upgrade plan. The fixes are ordered by impact-to-effort ratio — the highest-ROI fixes first.

| # | BUG                              | FIX                                        | EFFORT  | CHAPTER |
|---|----------------------------------|--------------------------------------------|---------|---------|
| 1 | Version label (3.8 vs 3.6)       | Verify actual model, update both locations | 5 min   | Ch 2    |
| 2 | Suggested prompts don't populate | Wire onClick → setInput + sendMessage      | 1 hour  | #39     |
| 3 | No conversation history          | Zustand + persist slice                    | 1 day   | Ch 6    |
| 4 | Mode patient = Mode pro          | Upgrade system prompt with MODE_PROMPTS    | 4 hours | Ch 3-4  |
| 5 | 11s batch render, no streaming   | Adopt Vercel AI SDK (streamText + useChat) | 1 day   | Ch 5    |

## 9.1 The Grade Trajectory



## 9.2 The Fix Priority Order

1. **Bug #1 (5 min)** — Fix version label. Verify whether the deployed model is 3.6 or 3.8, update both the Cmd-K label and the Copilot header.
2. **Bug #2 (1 hour)** — Wire suggested-prompt buttons. Add `onClick={() => { setMode(prompt.mode); setInput(prompt.text); sendMessage(prompt.text); }}`. Covered in #39.
3. **Bug #4 (4 hours)** — Upgrade system prompt. Add `MODE_PROMPTS` object with mode-specific instructions. Test Mode patient produces simplified output.
4. **Bug #5 (1 day)** — Adopt Vercel AI SDK. Replace `generateContent` with `streamText`. Replace client fetch with `useChat`. Add Stop button.
5. **Bug #3 (1 day)** — Add Zustand + persist slice. Wire Copilot to read/write from the store. Add «Effacer la conversation» button.
6. **Cost optimizations (2 days)** — Add history pruning, response caching, model selection, token budgeting.

### — CHAPTER 10 • FINAL ROADMAP & VERDICT

## The 2-Week Plan to Go From B to A+

The AI Copilot Core upgrade is the platform's **signature feature elevation**. In 2 weeks, it transforms the Copilot from a B-grade batch-render chatbot (innovative content, broken implementation) to an A+ streaming clinical assistant (progressive rendering, persistent conversations, mode-specific behavior, Darija-native, cost-optimized). This is the feature most likely to drive user adoption.

## 10.1 The Phased Plan

| PHASE                            | DURATION | FIXES                                                | GRADE AFTER |
|----------------------------------|----------|------------------------------------------------------|-------------|
| Phase 1 · Bug fixes              | 1 day    | #1 (version), #2 (prompts), #4 (mode patient)        | B+          |
| Phase 2 · Streaming              | 1 day    | #5 (Vercel AI SDK, useChat, Stop button)             | A           |
| Phase 3 · History                | 1 day    | #3 (Zustand + persist, «Effacer» button)             | A           |
| Phase 4 · Darija + cost          | 2 days   | Prompt upgrade (Darija few-shot), cost optimizations | A+          |
| Phase 5 · Server-side (after #8) | 1 week   | Thread list, export, search, branching               | A+          |

## 10.2 The ROI Summary

| INVESTMENT             | RETURNS                                                                                |
|------------------------|----------------------------------------------------------------------------------------|
| 5 minutes              | Version label fixed — trust in clinical output                                         |
| 1 hour                 | Suggested prompts work — onboarding friction eliminated                                |
| 4 hours                | Mode patient produces simplified output — patient audience served                      |
| 1 day                  | Streaming — 10× perceived latency improvement (11s → 1s)                               |
| 1 day                  | Conversation history — conversations survive refresh                                   |
| 2 days                 | Darija-native output + cost optimizations — market differentiator + 45% cost reduction |
| <b>Total: ~2 weeks</b> | <b>B → A+, 5 bugs fixed, 10× latency improvement, Darija-native, cost-optimized</b>    |

## 10.3 Why the Copilot Matters Most for Adoption

The Copilot is the feature most likely to drive user adoption. A pharmacist who tries DzPharm and asks the Copilot a question gets an immediate, structured, clinically-grounded response — that's the «wow moment» that converts a trial user into a repeat user. But today, that wow moment is undermined by 11s of staring at a spinner, dead-click suggested prompts, and a conversation that vanishes on refresh. Fix those 5 bugs, and the wow moment becomes **1-second first token, working shortcuts, persistent conversations, Darija-native output**. That's the difference between «interesting demo» and «I use this every day».

## 10.4 The Three Pillars Together

The three deep-dive technical design documents (#1 Routing, #8 Auth, #36 Copilot) form the platform's **three pillars**:

- ✓ #1 Routing (2 weeks) — the URL foundation. Unlocks 9 features, 380× SEO traffic. Makes the platform shareable, indexable, navigable.
- ✓ #8 Auth (4 weeks) — the identity foundation. Unlocks 6 features, closes P0 security hole, regulatory compliance. Makes the platform trustworthy, accountable, per-user.
- ✓ #36 Copilot (2 weeks) — the engagement foundation. Fixes 5 bugs, 10× latency improvement, Darija-native. Makes the platform's signature feature production-grade.

Together: **8 weeks of engineering, 20 features unlocked/improved, P0 security closed, regulatory compliance achieved, 380× SEO traffic, 10× Copilot latency improvement.** This is the platform transformation from «impressive demo» to «trustworthy clinical infrastructure with 190K monthly organic visitors and a production-grade AI assistant».

#### FINAL VERDICT · THE SIGNATURE FEATURE ELEVATED

The AI Copilot Core upgrade transforms the platform's signature feature from B-grade (innovative content, broken implementation) to A+ (streaming, persistent, mode-specific, Darija-native, cost-optimized). The 5-bug fix plan is 2 weeks of work: 1 day for bug fixes (#1, #2, #4), 1 day for streaming (#5), 1 day for history (#3), 2 days for Darija + cost optimization. The streaming upgrade (Vercel AI SDK) is the single highest-impact change — it reduces perceived latency from 11s to ~1s and matches the UX of ChatGPT. The system prompt upgrade is the highest-ROI change — it fixes Mode patient and adds Darija-native output at zero infrastructure cost. Together with #1 Routing and #8 Auth, this completes the platform's three-pillar transformation.

#### Immediate Action · Ship Phase 1 Today

1. Fix version label — verify deployed model, update both locations — 5 minutes.
2. Wire suggested-prompt onClick handlers — 1 hour.
3. Upgrade system prompt with MODE\_PROMPTS — 4 hours.
4. Verify Mode patient produces simplified output — 30 minutes.
5. Schedule Phase 2 (streaming via Vercel AI SDK) for tomorrow.
6. Schedule Phase 3 (history) and Phase 4 (Darija + cost) for the following week.

# A+

From B (5 bugs, 11s batch render, no history) to A+ (streaming, persistent, mode-specific, Darija-native, cost-optimized) in 2 weeks. The signature feature elevated to production-grade.

10

CHAPTERS

6

SCREENSHOTS

5

BUGS FIXED

10×

LATENCY IMPROVEMENT